



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

Khoa Điện – Điện tử

Bộ môn - Viễn Thông

Hệ thống phát hiện và cảnh báo té ngã thời gian thực tích hợp cảm biến, xử lý ảnh và định vị

*Real-time fall detection and alert system integrating sensors,
image processing, and positioning*

SV:

Trần Đức Hảo

GVHD:

PSG.TS Hà Hoàng Kha

Ngày 5 tháng 10 năm 2025

LỜI CẢM ƠN

Trước hết, tôi xin gửi lời biết ơn sâu sắc đến **PGS.TS. Hà Hoàng Kha**, người đã tận tâm hướng dẫn và đồng hành cùng tôi trong suốt quá trình thực hiện luận văn. Thầy đã chỉ bảo tận tình, nhắc nhở về thái độ học tập và khuyến khích tôi tiếp cận vấn đề một cách thực chất, giúp tôi tích lũy kiến thức và kỹ năng làm việc khoa học và bài bản.

Tôi cũng xin chân thành cảm ơn các thầy cô trong **Bộ môn Viễn Thông, Khoa Điện – Điện tử, Trường ĐH Bách khoa – ĐHQG TP.HCM** đã truyền đạt nền tảng kiến thức vững chắc và tạo điều kiện thuận lợi để tôi phát triển chuyên môn toàn diện.

Đặc biệt, tôi xin tri ân **gia đình** – nguồn động lực tinh thần vững chắc, luôn ủng hộ và khích lệ tôi trong suốt quá trình học tập và hoàn thành luận văn. Sự đồng hành của gia đình là điều quý giá nhất giúp tôi kiên trì và hoàn thành luận văn này.

Mục lục

- 1 I-GIỚI THIỆU
- 2 II-CƠ SỞ LÝ THUYẾT
- 3 III-THIẾT KẾ VÀ TRIỂN KHAI
- 4 IV-KẾT QUẢ THỰC NGHIỆM
- 5 KẾT LUẬN CHUNG

Mục lục chương: I-GIỚI THIỆU

- 1 I-GIỚI THIỆU
 - Vấn đề té ngã và tình hình nghiên cứu
 - Mục tiêu đặt ra và phạm vi thực hiện của đề tài

2 II-CƠ SỞ LÝ THUYẾT

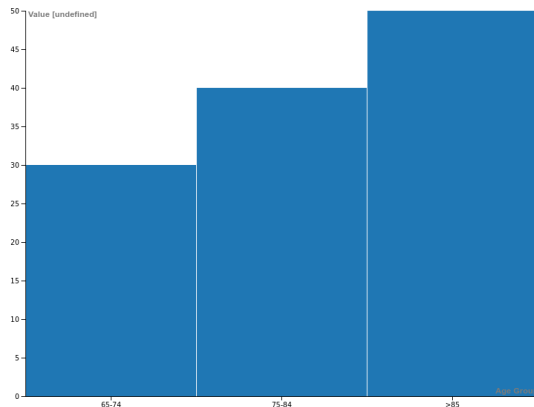
3 III-THIẾT KẾ VÀ TRIỂN KHAI

4 IV-KẾT QUẢ THỰC NGHIỆM

5 KẾT LUẬN CHUNG

Té ngã: Vấn đề chung toàn cầu

- Nguyên nhân chính gây chấn thương và tử vong không cố ý.
- **WHO**: ~ 646,000 ca tử vong/năm;
> 80% ở các nước thu nhập trung bình/thấp.
- Người cao tuổi: **30%** té ngã/năm ở người > 65 tuổi, tăng lên **50%** ở người > 85 tuổi.



Hình: Tỷ lệ té ngã theo nhóm tuổi

khái quát hiện trạng nghiên cứu cứu trong và ngoài nước

Quốc tế

- **Xu hướng:** AI nhẹ (TinyML), YOLO, Transformer, cảm biến mmWave
- **Thành tựu:** giảm cảnh báo sai, tối ưu cho thiết bị biên, kết hợp dữ liệu đa cảm biến.

Trong nước

- **Thực trạng:** chủ yếu mô hình thử nghiệm với ESP32, Arduino, Python
- **Hạn chế:** thiếu dữ liệu lớn, độ chính xác chưa cao.

Các phương pháp phát hiện té ngã

Dựa trên thị giác

Sử dụng camera và thuật toán nhận diện tư thế người (MediaPipe, OpenPose)

Dựa trên cảm biến đeo

Sử dụng cảm biến quán tính (IMU, MPU6050) trên thiết bị

Kết hợp đa phương thức

Tích hợp dữ liệu từ nhiều nguồn để tăng độ chính xác

Ưu nhược điểm các phương pháp phát hiện té ngã

Phương pháp	Cơ chế	Ưu điểm	Nhược điểm
Đeo được	IMU (gia tốc kế, con quay hồi chuyển); phát hiện gia tốc/tư thế bất thường	Phản hồi nhanh; chính xác; chi phí thấp	Cần đeo liên tục; dễ false positive; thời lượng pin giới hạn
Môi trường	Cảm biến cố định: sàn áp suất, PIR, âm thanh; AI phân tích	Không xâm phạm; giám sát nhiều người; tích hợp smart home	Chi phí cao; phạm vi hạn chế; nhầm vật thể
Thị giác	Camera RGB/RGB-D/IR; pose estimation (OpenPose/MediaPipe)	Thông tin trực quan; không cần đeo; tích hợp giám sát	Quyền riêng tư; phụ thuộc ánh sáng; cần phần cứng mạnh
Đa phương thức	Kết hợp IMU + camera + môi trường; data fusion (Kalman/Deep Learning)	Độ chính xác cao; giảm cảnh báo sai; mở rộng phạm vi; kinh tế	Phức tạp; tốn năng lượng; đồng bộ khó

Mục tiêu và giới hạn của sản phẩm

Mục tiêu hiệu năng

- ✓ Tổng độ trễ < 5 giây.
- ✓ Độ chính xác $> 90\%$, False Alarm $< 8\%$.
- ✓ Uptime dịch vụ MQTT $> 99\%$.

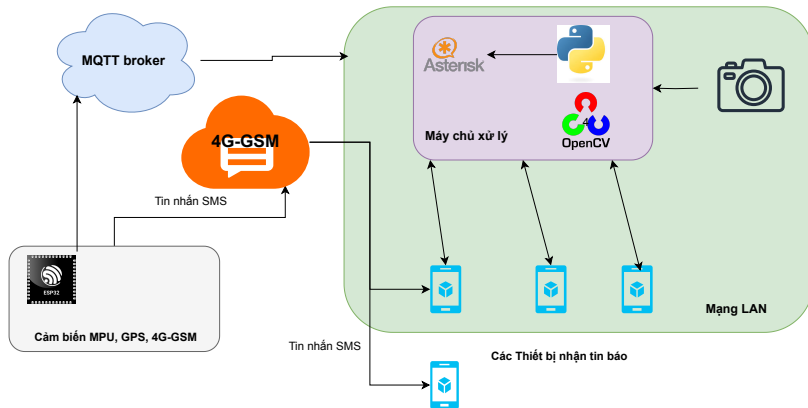
Lựa chọn đa phương thức

Kết hợp dữ liệu từ nhiều cảm biến giúp hệ thống giám sát rộng hơn và nhận diện té ngã linh hoạt hơn.

Giới hạn sản phẩm

- × Hoạt động trong nhà với điều kiện ánh sáng và mạng ổn định.
- × Nguyên mẫu chưa tích hợp học sâu toàn phần.
- × Không phát triển app di động/web phức tạp.

Sơ đồ cấu trúc hệ thống



Hình: Sơ đồ cấu trúc hệ thống

Mục lục chương: II-CƠ SỞ LÝ THUYẾT

1 I-GIỚI THIỆU

2 II-CƠ SỞ LÝ THUYẾT

- Các phương thức truyền thông áp dụng trong đề tài
- Thị giác máy tính và nhận diện tư thế người
- Cơ sở lý thuyết xây dựng phần cứng

3 III-THIẾT KẾ VÀ TRIỂN KHAI

4 IV-KẾT QUẢ THỰC NGHIỆM

5 KẾT LUẬN CHUNG

Giao thức SIP - Khởi tạo Phiên và Đường tín hiệu/Media

Chức năng chính của SIP trong hệ thống

- Thiết lập và quản lý cuộc gọi VoIP từ hệ thống cảnh báo.
- **Asterisk PBX** điều phối liên lạc sofphone nội mạng
- Truyền âm thanh cảnh báo qua **RTP**.

Quy trình bắt tay hoạt động cơ bản

REGISTER → INVITE → ACK → RTP → BYE

Đường tín hiệu (Signaling Path)

- Mang bản tin SIP: INVITE, BYE, ...
- Thiết lập, quản lý, kết thúc cuộc gọi.
- Chạy trên **TCP/UDP**.

Đường truyền phương tiện (Media Path)

- Mang dữ liệu thoại/video thực tế.
- Sử dụng **RTP qua UDP**.
- Truyền trực tiếp giữa các điểm cuối.

SIP – ICE và Asterisk

NAT / Firewall

- Thiết bị VoIP thường ở sau NAT/Firewall.
- Cản trở truyền **RTP trực tiếp**.

ICE (Interactive Connectivity Establishment)

- **Local IP**: địa chỉ nội bộ.
- **STUN**: phát hiện IP công cộng/cổng NAT.
- **TURN**: chuyển tiếp media khi STUN thất bại.



Asterisk PBX

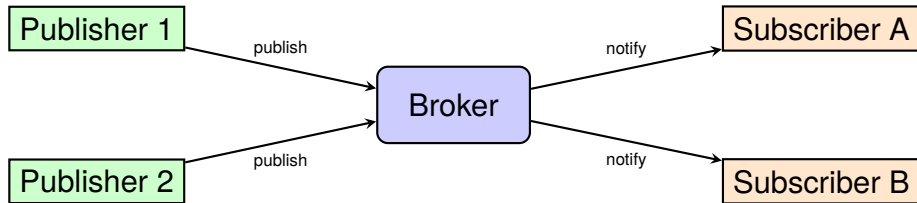
- Quản lý và định tuyến cuộc gọi VoIP.
- Chuẩn mở SIP, bảo mật **TLS/SRTP**.
- Đăng ký SIP và hỗ trợ nhiều thiết bị.

MQTT: Tổng quan và Kiến trúc Pub/Sub

Định nghĩa & Đặc điểm

- **MQTT**: Message Queuing Telemetry Transport
- Giao thức **nhẹ**, tối ưu cho IoT/M2M
- Hoạt động trên **TCP/IP**, kết nối lâu dài
- Thích hợp cho thiết bị **tài nguyên hạn chế**, mạng **băng thông thấp**

Kiến trúc Publish/Subscribe



MQTT: QoS, Lợi ích và Bảo mật

QoS Levels

- **QoS 0** – Fire & Forget, nhanh, có thể mất bản tin.
- **QoS 1** – At least once, đảm bảo nhận, có thể trùng.
- **QoS 2** – Exactly once, tin cậy nhất nhưng chậm.

Lợi ích

- Truyền/nhận tách biệt (Pub–Sub).
- Giảm độ trễ, hỗ trợ offline.
- Dễ mở rộng hệ thống.

Bảo mật trong MQTT

- **Mã hoá kênh:** TLS/SSL (port 8883), certificate pinning.
- **Xác thực:** Username/Password, Client certificates, Token (JWT, OAuth2).
- **Uỷ quyền:** ACL theo topic, RBAC cho nhóm người dùng.
- **Bảo vệ hệ thống:** hạn chế kết nối, rate limiting, chống DoS.

JSON - Định nghĩa, Cấu trúc và Thư viện

Định nghĩa cấu trúc:

- Định dạng nhẹ, độc lập ngôn ngữ
- Key:value, Object, Array, String, Number, Boolean, null
- Ứng dụng: lưu cấu hình, payload MQTT, API, cảnh báo

Thư viện nhúng:

- json-c: C cho C/C++
- FirebaseJson: dễ dùng cho IoT

Ví dụ JSON

```
1 {  
2   "device_id": "ESP32_001",  
3   "temperature": 25.5,  
4   "sensors": ["temp", "light"]  
5 }
```


Tối Ưu Hóa và Kết Hợp SIP/MQTT/JSON

Vai trò từng giao thức:

- **MQTT**: Truyền dữ liệu IoT hiệu quả, hỗ trợ QoS.
- **JSON**: Định dạng linh hoạt, dễ tích hợp hệ thống.
- **SIP**: Cảnh báo âm thanh/thoại tức thì.

Hướng tối ưu:

- Payload JSON nhỏ gọn, giảm băng thông.
- Chọn mức QoS phù hợp cho từng loại dữ liệu.
- Tự động reconnect, đảm bảo tính sẵn sàng.
- **Bảo mật**: TLS + xác thực người dùng.

Tích hợp đa giao thức = **Hiệu quả + Độ tin cậy + An toàn**

Định nghĩa và ứng dụng của computer vision

Định nghĩa

Thị giác Máy tính (CV): Lĩnh vực AI cho phép máy tính xử lý, phân tích và diễn giải hình ảnh/video, mô phỏng thị giác con người.

Mục tiêu

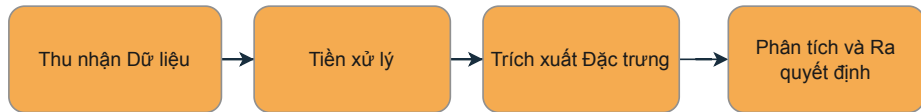
- Tái tạo khả năng nhận thức thị giác với tốc độ, độ chính xác và quy mô vượt trội.
- Ứng dụng trong Hệ thống phát hiện và cảnh báo té ngã thời gian thực tích hợp cảm biến, xử lý ảnh và định vị, đặc biệt là **Ước lượng Tư thế Người (HPE)**.

Pipeline Cơ bản của Hệ thống CV

Quy trình

- 1 **Thu nhận dữ liệu:** Thu thập ảnh/video từ camera.
- 2 **Tiền xử lý:** Chuẩn hóa kích thước, điều chỉnh sáng/tương phản, giảm nhiễu.
- 3 **Trích xuất đặc trưng:** Chuyển pixel thành đặc trưng trừu tượng (cạnh, góc, kết cấu).
- 4 **Phân tích và quyết định:** Phân loại, nhận dạng hoặc ước lượng tư thế.

Minh họa



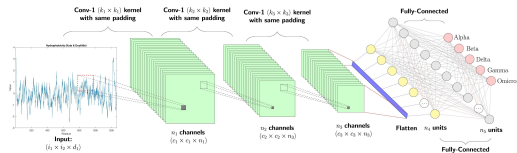
Quy trình tổng thể của hệ thống CV.

Mô hình Học sâu: CNN (Convolution Neural Network)

- **Mạng Nơ-ron Tích chập (CNN):** Kiến trúc chủ đạo cho xử lý ảnh.
- **Phép tích chập:** Trích xuất đặc trưng cục bộ từ vùng nhỏ của ảnh.

$$(I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n) \quad (1)$$

- I : ma trận ảnh đầu vào.
- K : kernel (bộ lọc).
- (i, j) : vị trí pixel trong đầu ra.
- **Phép gộp (Pooling):** Giảm kích thước, tăng tính bền vững.



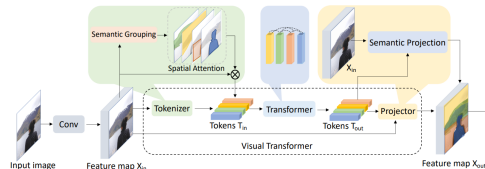
Hình: Phép tích chập và gộp.

Mô hình Học sâu: ViT (Vision Transformer)

- **ViT**: Chia ảnh thành **patch** (miếng vá), xử lý như chuỗi token.
- **Cơ chế Self-Attention**: Tính trọng số quan hệ giữa các token.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2)$$

- Q, K, V : ma trận truy vấn, khóa, giá trị.
- d_k : độ dài vector khóa (chuẩn hóa).
- Giúp học **quan hệ toàn cục**, vượt giới hạn cục bộ của CNN.



Hình: Kiến trúc ViT.

Các Bài toán Chính trong CV

Các nhóm bài toán thường gặp

- **Phân loại Ảnh (Image Classification)** → Đầu ra: một nhãn duy nhất cho cả bức ảnh. *Ví dụ:* ảnh chứa người hay xe.
- **Phát hiện Đối tượng (Object Detection)** → Đầu ra: nhãn + vị trí (hộp giới hạn bounding box). *Ví dụ:* phát hiện nhiều người trong cùng một khung hình.
- **Phân đoạn Ảnh (Image Segmentation)**
 - **Ngữ nghĩa (Semantic):** gán nhãn từng pixel theo lớp. *Ví dụ:* tất cả pixel thuộc “Đường”, “Cây”.
 - **Thể hiện (Instance):** phân biệt từng cá thể trong cùng lớp. *Ví dụ:* tách riêng từng chiếc xe thay vì gom tất cả thành “Xe”.
- **Ước lượng Tư thế Người (Human Pose Estimation - HPE)** → Đầu ra: tọa độ các khớp xương (keypoints) của cơ thể. *Ứng dụng:* phân tích chuyển động, giám sát té ngã.

Ứng dụng CV trong Nhận diện Tư thế Người và Phát hiện Té ngã

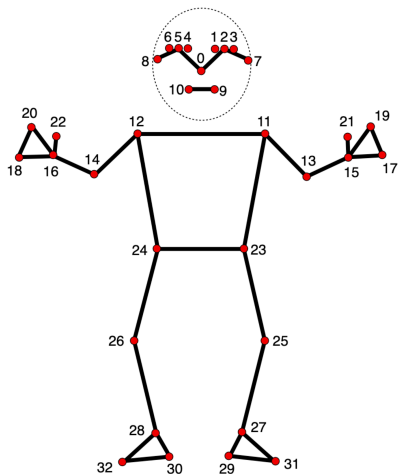
Mục tiêu

Xác định tư thế cơ thể người theo thời gian thực và phát hiện sự kiện té ngã để hỗ trợ giám sát an toàn.

Phương pháp

- ➊ **Nhận diện tư thế:** Dùng thư viện Python **MediaPipe Pose** để trích xuất **33 điểm khớp** (keypoints) trên cơ thể.
- ➋ **Đặc trưng động học/tư thế:** Tính toán góc, vận tốc, và sự thay đổi đột ngột về vị trí cơ thể.
- ➌ **Phát hiện té ngã:** Kết hợp logic ngưỡng (thresholds) và thuật toán học máy (ML) để phân loại té ngã.

Keypoints



Khái niệm

Ước lượng vị trí khớp từ hình ảnh/video:

$$\mathcal{K} = \{k_i = (x_i, y_i, z_i, c_i)\}$$

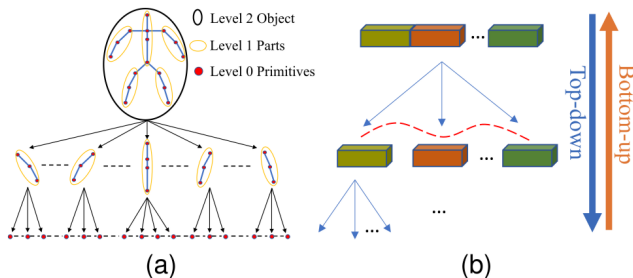
(c_i là confidence score cho mỗi keypoint).

Phương pháp

- **Top-down:** Phát hiện người trước, sau đó keypoints (MediaPipe).
- **Bottom-up:** Keypoints trước, nhóm thành người sau (OpenPose).

Keypoints cơ bản trong HPE (Human

Mô hình nhận dạng đối tượng phân cấp



Hình: *

Minh họa cấu trúc phân cấp đối tượng

Cấu trúc phân cấp:

- **Primitives (Cấp 0)**: Thành phần cơ bản (điểm, đường, góc).
- **Parts (Cấp 1)**: Tập hợp các Primitives tạo bộ phận.
- **Object (Cấp 2)**: Tập hợp Parts tạo đối tượng hoàn chỉnh.

Đặc trưng động học và phát hiện té ngã

Đặc trưng Động học

- **Vận tốc COM:** $\vec{v}_{\text{COM}} = \frac{\Delta \vec{p}}{\Delta t}$
- **Gia tốc:** $a = \frac{\|\Delta \vec{v}\|}{\Delta t}$

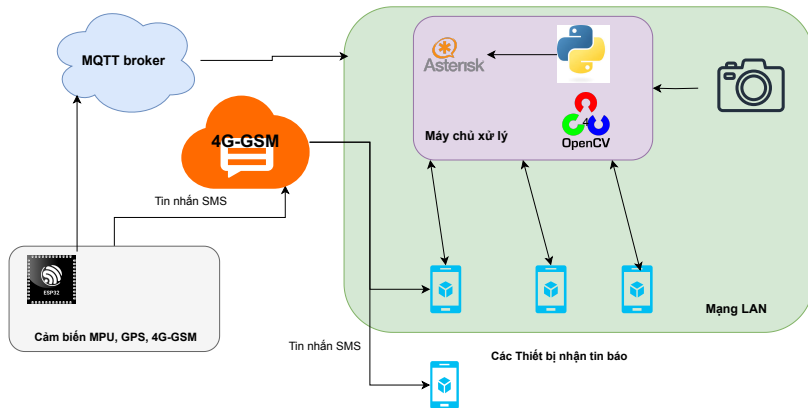
Đặc trưng Tư thế

- **AR (Aspect Ratio):** Tăng khi người nằm ngang
 - θ_{body} : Góc vai-hông
 - Δh_{head} : Giảm chiều cao đầu
- (AR, θ , Δh giúp xác định tư thế bất thường).*

Ba Giai đoạn Phát hiện

- 1 **Sớm:** Tốc độ/gia tốc COM cao
- 2 **Xác nhận:** AR, θ_{body} chỉ nằm ngang
- 3 **Bất động:** Chuyển động $< M_{th}$

Sơ đồ thực nghiệm



Hình: Sơ đồ cấu trúc thực nghiệm

Môi trường Phát triển (ESP-IDF) triển khai trên nền linux

Đặc trưng của ESP-IDF

- **Build system CMake + Kconfig** – cấu hình linh hoạt, dễ mở rộng component
- **FreeRTOS tích hợp sẵn** – quản lý đa nhiệm giả lập tác vụ song song
- **Driver cấp thấp** – I2C, SPI, UART, PWM, GPIO được tối ưu cho ESP32
- **Hỗ trợ mạng phong phú** – Wi-Fi, Bluetooth, TCP/IP stack, MQTT, HTTP(S)

Lợi ích cho hệ thống Phát hiện Té ngã

- Quản lý **đa component** (IMU, SIM4G, LED, Fall Logic) độc lập, dễ khoanh vùng lỗi, tái sử dụng và mở rộng project
- Thực thi **song song**: đồng thời xử lý nhiều luồng giúp tối ưu thời gian phản hồi của thiết bị
- Debug chuyên nghiệp: gdbstub, logging

Mục lục chương: III-THIẾT KẾ VÀ TRIỂN KHAI

1 I-GIỚI THIỆU

2 II-CƠ SỞ LÝ THUYẾT

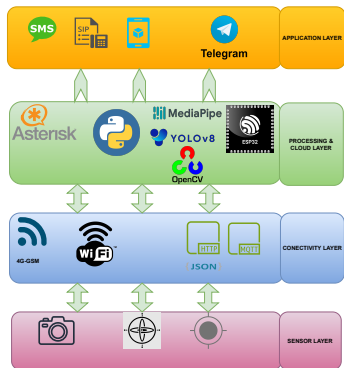
3 III-THIẾT KẾ VÀ TRIỂN KHAI

- Tổng quan kiến trúc FDAS(Fall detection and alert system)
- Thực hiện phần cứng của FDAS
- Triển khai thực hiện phần mềm

4 IV-KẾT QUẢ THỰC NGHIỆM

5 KẾT LUẬN CHUNG

Kiến trúc theo phân lớp của FDAS



Quy trình

- Minh họa kiến trúc hệ thống phát hiện té ngã và cảnh báo.

Mô tả

- **Lớp Thiết bị/Biên:** Thu thập dữ liệu.
- **Lớp Kết nối:** Truyền tải dữ liệu.
- **Lớp Xử lý/Đám mây:** Xử lý và ra quyết định.
- **Lớp Ứng dụng/UI:** Giao diện người dùng.

Tổng quan dựa trên phần cứng của FDAS

1. Nguồn dữ liệu

- **Thiết bị đeo / Cảm biến:** IMU, ESP32, 4G/GPS, thu thập chuyển động định vị.
- **Camera giám sát:** Xác nhận sự kiện té ngã qua hình ảnh.

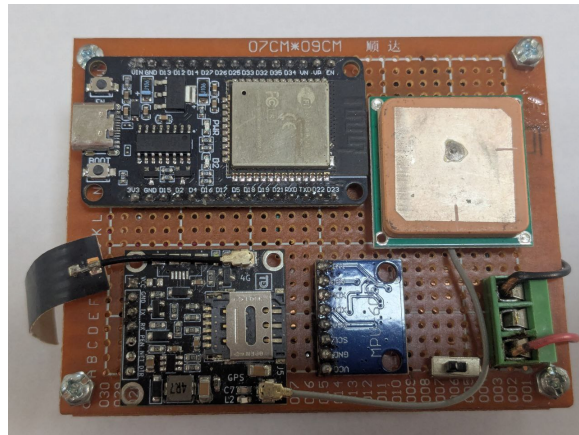
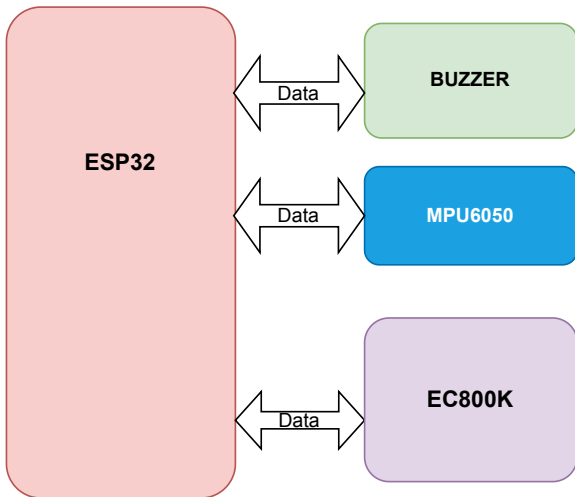
2. Thành phần chính

- Máy chủ/Xử lý: Phân tích dữ liệu, Học sâu
- Truyền thông: Wi-Fi, 4G/LTE đảm bảo kết nối

3. Ưu điểm & Nguyên lý

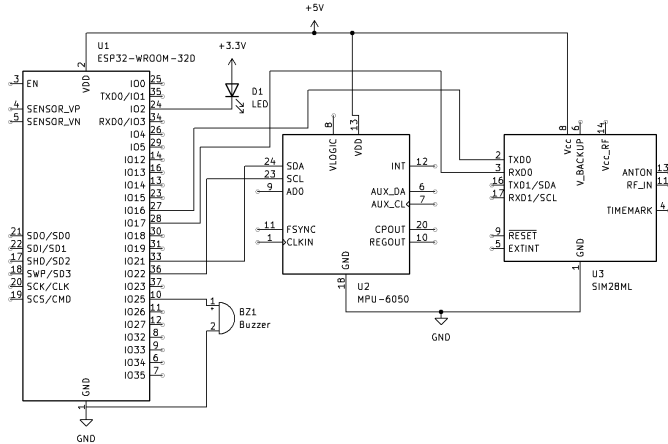
- Linh hoạt triển khai, giám sát phạm vi rộng
- Duy trì hoạt động khi 1 module lỗi
- Hai nguồn dữ liệu độc lập tăng độ tin cậy phát hiện

Module I: Thiết bị đeo/Cảm biến



Hình: Module I thực tế

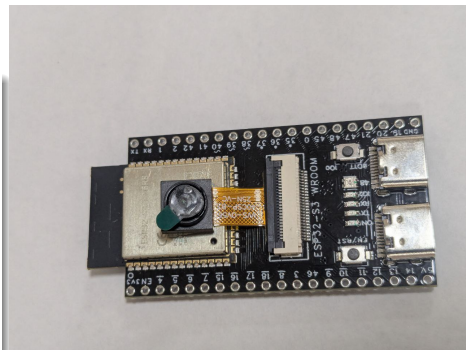
Schematic module I



Module II: Camera giám sát

Thành phần chính

- **ESP32-S3-N16R8:**
 - Tích hợp PSRAM
 - Giao diện camera chuyên dụng
- **Module II: Camera OV5640:**
 - Cảm biến 5MP
 - Đa kích thước (QQVGA-UXGA)
 - Bus 8-bit, 20MHz



Hình: Module II thực tế

Tổng quan các Phần mềm cần thực hiện của toàn bộ Hệ thống

Các thành phần chính

#	Thành phần	Nền tảng	Công nghệ chính	Chức năng
1	Asterisk	Linux Mint 21	PJSIP, AMI	Hệ thống VoIP
2	Module I	ESP32	ESP-IDF, C/C++	Cảm biến đeo, phát hiện ngã
3	Module II	ESP32	ESP-IDF, OpenCV	Camera giám sát
4	Python Server	Linux	Python, MQTT	Xử lý trung tâm, AI

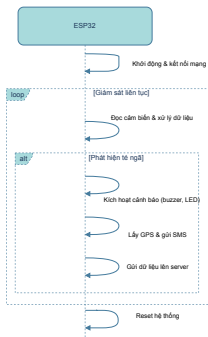
Asterisk: Cấu hình Ba Thành Phần Chính

Thành phần	Vai trò	Điểm chính
PJSIP (<i>SIP Stack</i>)	Quản lý kết nối từ điện thoại IP, xác thực người dùng trước khi cho phép kết nối	Tạo tài khoản cho thiết bị, định dạng âm thanh truyền tải
Dial Plan (<i>Kịch bản xử lý</i>)	Định nghĩa luồng xử lý khi có cuộc gọi hoặc tin nhắn, thực thi các lệnh gọi điện	Quy tắc gọi trực tiếp, gọi nhóm, và xử lý tin nhắn SIP
AMI (<i>Giao diện quản lý</i>)	Cho phép ứng dụng Python điều khiển Asterisk từ xa (gọi, ngắt máy, lấy trạng thái)	Kết nối qua mạng nội bộ với xác thực username/password

Luồng xử lý tin báo

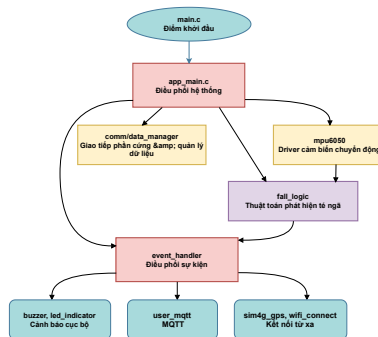
Python Server $\xrightarrow{\text{AMI}}$ Asterisk $\xrightarrow{\text{Dial Plan}}$ Thực hiện cuộc gọi $\xrightarrow{\text{PJSIP}}$ Điện thoại IP

Module cảm biến đeo I: Luồng làm việc & Phần mềm



Hình: *

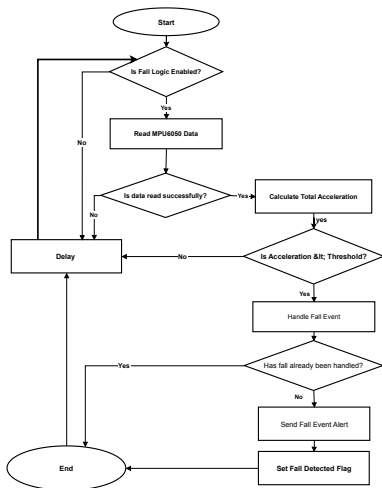
Luồng xử lý từ cảm biến đến cảnh báo.



Hình: *

Sơ đồ khối phần mềm ESP32.

Logic Phát hiện Té ngã (Component fall_logic)



Các Giai đoạn Phát hiện Té ngã

Giai đoạn 1: STATE_LOW_G (Rơi tự do)

Điều kiện: $Accel < 0.3g$, Ý nghĩa: Cơ thể rơi tự do / mất trọng lượng

Giai đoạn 2: STATE_HIGH_G (Va chạm)

Điều kiện: $Accel > 3.0g$, Ý nghĩa: Xác nhận va chạm đột ngột

Giai đoạn 3: STATE_CONFIRMED (Bất động)

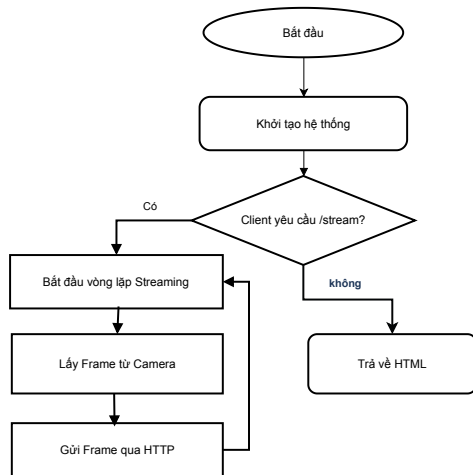
Điều kiện: $Accel$ ổn định $0.8 - 1.2g$ kéo dài $> 10s$, Ý nghĩa: Kích hoạt cảnh báo té ngã

Mục tiêu: Giảm False Positives bằng cách yêu cầu đủ 3 điều kiện theo thứ tự: rơi → va chạm → bất động

Module Camera: Tổng Quan & Luồng Hoạt Động

Luồng hoạt động

- 1 **Khởi tạo:** Cấu hình Wi-Fi, camera, HTTP server
- 2 **Chờ kết nối:** Lắng nghe yêu cầu client
- 3 **Lấy frame:** Đọc khung hình từ buffer
- 4 **Truyền tải:** Gửi JPEG qua HTTP multipart
- 5 **Giám sát:** Theo dõi FPS và xử lý lỗi



Python Server: Kiến trúc Luồng Xử lý

Kiến trúc phân tầng

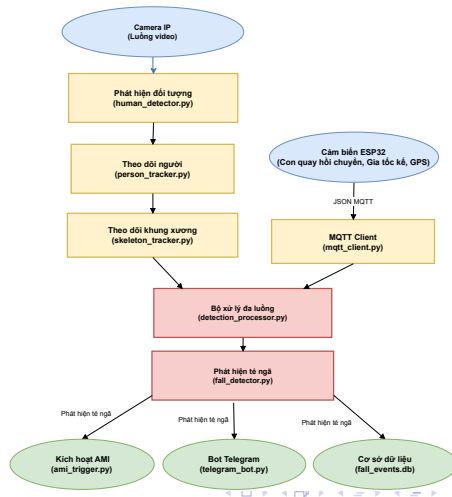
Thu nhận: comm/ (MQTT, Telegram, AMI), detection/ (video, tracking)

Xử lý: fall/, processing/

Đầu ra: database/, cảnh báo

Luồng Xử lý

1. Thu nhận: ESP32 (MQTT), Camera IP (AI)
2. Tích hợp: DetectionProcessor đồng bộ dữ liệu
3. Đầu ra: Lưu sự kiện, gửi cảnh báo qua AMI/Telegram



Python Server: Thuật toán Phát hiện Té ngã

Quy trình trong FallDetector

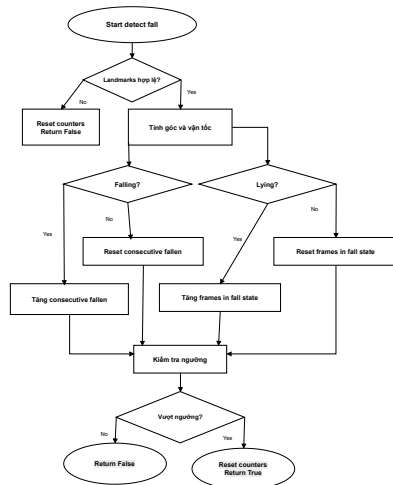
1. Kiểm tra Landmark: - Đảm bảo các điểm quan trọng (vai, hông) hợp lệ và đủ độ tin cậy

2. Tính toán Torso & Velocity: - Góc thân: dọc & ngang
- Vận tốc trung tâm torso giữa các frame

3. Xác định trạng thái: - *Falling*: góc vượt ngưỡng + vận tốc > threshold

- *Lying*: góc vượt ngưỡng kéo dài
- Cập nhật counters: consecutive_frames_fallen, frames_in_fall_state

4. Quyết định té ngã: - Nếu counters $\geq$ threshold $\rightarrow$ xác nhận té ngã, reset trạng thái



Mục tiêu

Phát hiện ngã đa hướng (trước/sau/ngang), giảm tối đa False Positives

Mục lục chương: IV-KẾT QUẢ THỰC NGHIỆM

1 I-GIỚI THIỆU

2 II-CƠ SỞ LÝ THUYẾT

3 III-THIẾT KẾ VÀ TRIỂN KHAI

4 IV-KẾT QUẢ THỰC NGHIỆM

- Thiết lập và kết quả thực nghiệm
- Kiểm thử từng thành phần

5 KẾT LUẬN CHUNG

Thiết lập thực nghiệm

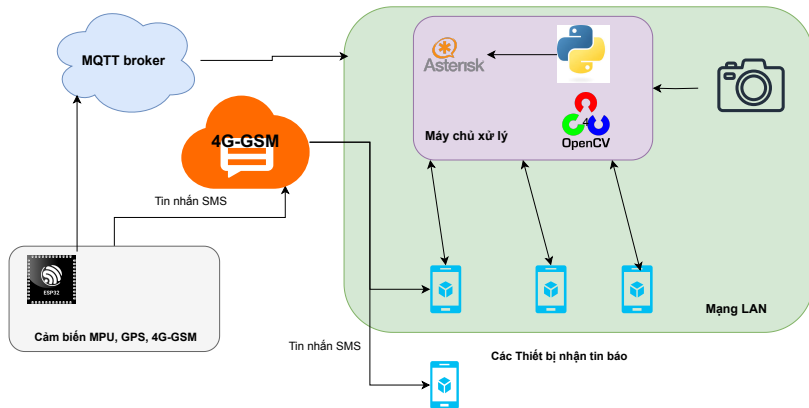
Môi trường thử nghiệm

- Trong nhà, điều kiện lý tưởng
- **Độ chính xác**: Phát hiện đúng té ngã
- **Độ trễ**: Thời gian cảnh báo
- **Độ ổn định**: Vận hành liên tục

Cấu hình hệ thống

Thành phần	Thông số
Máy chủ	Intel i7-2630QM, GT 525M, Linux Mint
Phần mềm	Asterisk 22.4, Python 3.10.13
Cảm biến đeo	ESP32 + SIM, Wi-Fi/GSM
Camera	ESP32-S3 + webcam dự phòng
Di động	Smartphone + Linphone 6.0.17

Sơ đồ thực nghiệm



Hình: Sơ đồ cấu trúc thực nghiệm

Kiểm thử Module 4G/GPS & Khởi tạo hệ thống

```
1 I (2329) SIM_4G: Received: Quectel EG800K
2 OK
3 I (5329) SIM_4G: Received: +CSQ: 31,99
4 OK
5 I (28339) SIM_4G: Received: +QIACT:
6   ↪ 1,1,1,"9.204.251.200"
7 OK
8 I (34349) SIM_4G: Received: +QGPSLOC:
9   ↪ 10.88862,106.77975
10 OK
11 I (9961) SIM4G_AT: Initializing SIM4G AT
   ↪ driver ...
I (10011) APP_MAIN: System initialization
   ↪ complete.
I (10021) APP_MAIN: Application started
   ↪ successfully
```

• Mục đích:

- Xác nhận giao tiếp AT command, kết nối 4G và thu nhận GPS.
- Kiểm tra, module SIM4G-GPS, Wi-Fi và các tác vụ chính.

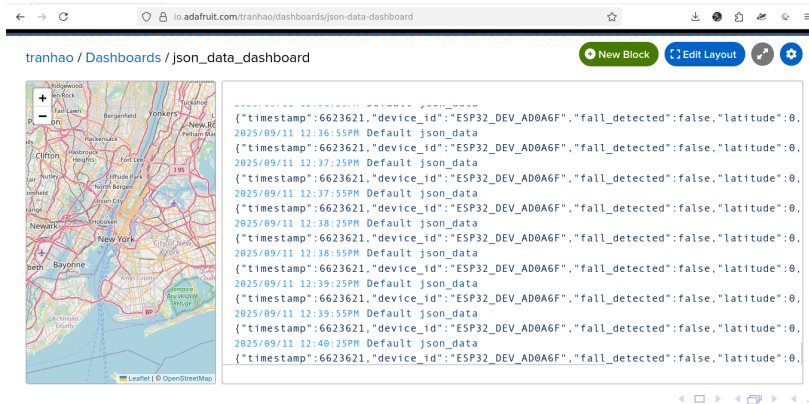
• Quy trình:

- 1 Khởi tạo UART, kiểm tra modem và SIM.
- 2 Đánh giá chất lượng sóng (+CSQ).
- 3 Cấu hình APN
- 4 Bật GPS và truy vấn tọa độ.

• Kết luận:

Kiểm thử MQTT và truyền dữ liệu

- **Mục đích:** Kiểm tra kết nối broker MQTT và gửi bản tin định kỳ.
- **Kết quả** Kết nối MQTT ổn định, dữ liệu được truyền tới hệ thống giám sát.



The screenshot shows the Adafruit IO web interface. The browser address bar displays `io.adafruit.com/tranhao/dashboards/json-data-dashboard`. The page title is `tranhao / Dashboards / json_data_dashboard`. The dashboard contains two main components:

- Map:** A map of New York City, showing various neighborhoods like Manhattan, Bronx, and Queens. A red location pin is visible in the Bronx area.
- JSON Data Stream:** A list of JSON objects representing sensor data. Each object contains the following fields: `timestamp`, `device_id`, `fall_detected`, and `latitude`. The data is as follows:

```
{ "timestamp": "2025/09/11 12:36:55PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
{ "timestamp": "2025/09/11 12:37:25PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
{ "timestamp": "2025/09/11 12:37:55PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
{ "timestamp": "2025/09/11 12:38:25PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
{ "timestamp": "2025/09/11 12:38:55PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
{ "timestamp": "2025/09/11 12:39:25PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
{ "timestamp": "2025/09/11 12:39:55PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
{ "timestamp": "2025/09/11 12:40:25PM", "device_id": "ESP32_DEV_AD0A6F", "fall_detected": false, "latitude": 0, "default_json_data": true }
```

Phát hiện Té ngã & Xử lý Cảnh báo

Quy trình

- Ghi nhận chuỗi trạng thái: LOW_G → HIGH_G.
- Kích hoạt cảnh báo: SMS, MQTT, buzzer, LED.

Log thực tế

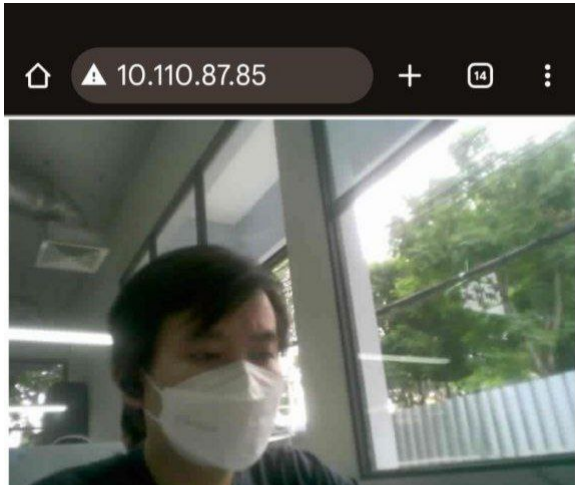
```

1 E (159131) FALL_LOGIC: FALL DETECTED! Accel:
   ↳ 0.99 g
2 I (159151) SIM4G_GPS: SMS request queued
   ↳ successfully
3 I (159881) SIM4G_AT: SMS sent successfully.
  
```



Hình: Log cảm biến khi phát hiện té ngã

Camera ESP32-CAM: Luồng hình ảnh và xử lý

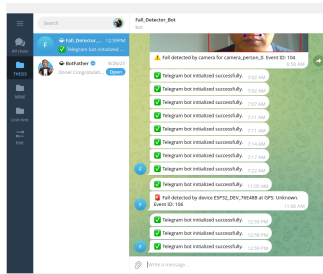


- **Mục đích:** Kiểm tra kết nối Wi-Fi và phát luồng hình ảnh.
- **Kết quả:** Thời gian thực 3–5 FPS, hoạt động ổn định.

Telegram Bot: Cảnh báo té ngã từ ESP32 và Python

Module Phần cứng (ESP32 + 4G/GPS):

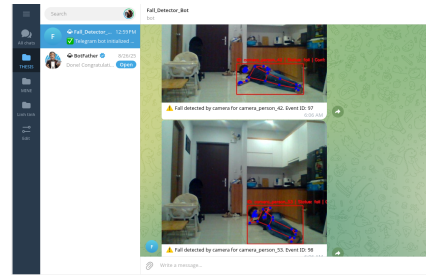
- Khi phát hiện té ngã, bản tin MQTT gửi tới hệ thống trung tâm
- Hệ thống trung tâm chuyển tiếp thông tin tới Telegram cho người dùng



Hình: Cảnh báo từ ESP32

Module Python + Camera:

- Phát hiện té ngã từ video và skeleton
- Gửi trực tiếp cảnh báo Telegram kèm hình chụp và thông tin chi tiết



Hình: Cảnh báo từ Python

Đánh giá kết quả thực nghiệm

Độ trễ hệ thống (Latency)

- **ESP32**: đọc cảm biến, xử lý và phát cảnh báo.
- **Mạng**: truyền gói MQTT tới broker.
- **Dịch vụ**: Python nhận dữ liệu, kích hoạt bot Telegram.
- **Tổng thể**: độ trễ < 5 giây, đáp ứng gần thời gian thực.

Đánh giá độ ổn định & hiệu năng

- Hệ thống hoạt động liên tục, không tràn bộ nhớ hay reset đột ngột.
- Thuật toán phân biệt rõ trạng thái bình thường và té ngã.
- Dữ liệu phối hợp chính xác, phần mềm Python ổn định.
- Giật/lag do phần cứng, không ảnh hưởng tổng thể.

Mục lục chương: KẾT LUẬN CHUNG

- 1 I-GIỚI THIỆU
- 2 II-CƠ SỞ LÝ THUYẾT
- 3 III-THIẾT KẾ VÀ TRIỂN KHAI
- 4 IV-KẾT QUẢ THỰC NGHIỆM
- 5 KẾT LUẬN CHUNG**

Kết quả chung & Hướng phát triển

Kết quả chính

- Hệ thống IoT tích hợp hoàn chỉnh từ cảm biến - xử lý - thiết bị cuối
- Thuật toán phát hiện té ngã: kết hợp dữ liệu cảm biến và hình ảnh theo thời gian thực.
- Cơ chế cảnh báo đa dạng: buzzer, LED, SMS, MQTT/Telegram, SIP.
- Thực nghiệm: hệ thống hoạt động với độ chính xác tương đối, phản hồi nhanh
- Tiềm năng ứng dụng: giám sát sức khỏe từ xa, ứng dụng trong các trung tâm chăm sóc sức khỏe.

Hạn chế & Hướng phát triển

- Quy mô thử nghiệm còn hạn chế; số lượng thiết bị đầu cuối còn hạn chế
- Thuật toán chưa hoàn toàn ổn định; thiết bị đeo hiện tại còn cồng kềnh.
- Kết nối 4G/GPS chưa tối ưu, độ tin cậy cần cải thiện.
- **Hướng phát triển:** mở rộng quy mô thử nghiệm, cải tiến thuật toán, thu gọn kích thước thiết bị, hoàn thiện kết nối 4G/GPS, tích hợp nền tảng IoT.

Kết luận chung & Lời cảm ơn

- Nguyên mẫu FDAS vận hành thành công: phát hiện té ngã và gửi cảnh báo trong thời gian thực.
- Độ trễ đầu-cuối < 5 giây; hệ thống ổn định trong điều kiện thử nghiệm
- Đề tài đã cung cấp kiến thức thực tiễn về IoT, nhúng, sensor fusion, và hệ thống phân tán.
- Xác định các thách thức thực tế: tối ưu phần cứng, nâng cao độ chính xác thuật toán, đảm bảo truyền thông ổn định.

Cảm ơn quý thầy/cô và các bạn đã theo dõi!